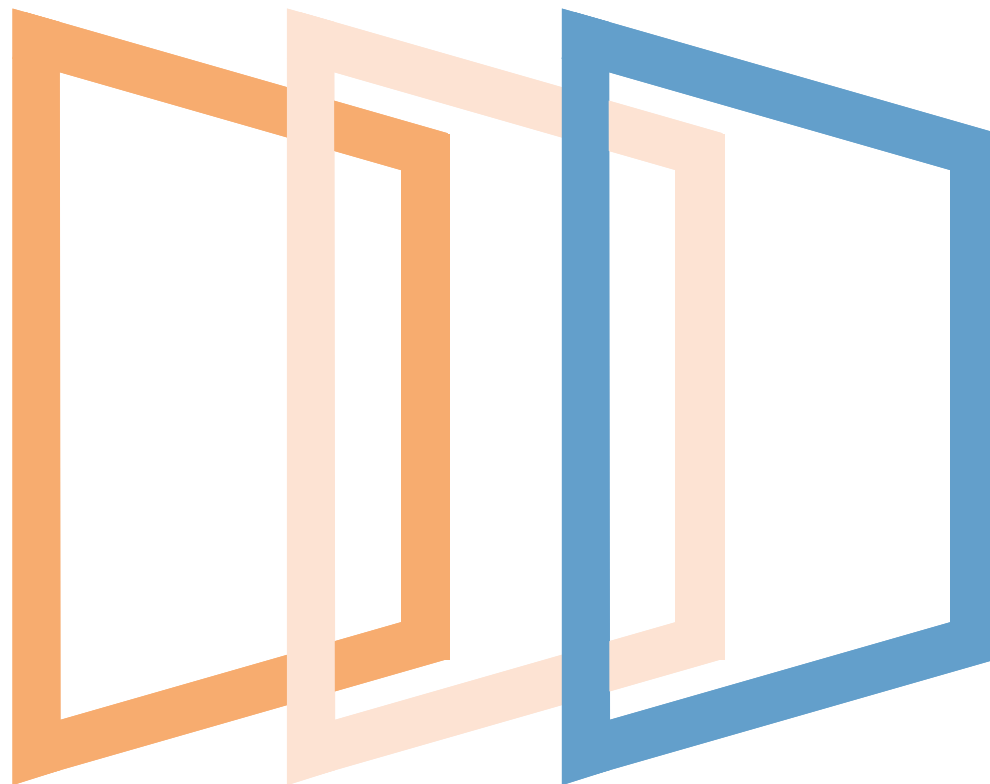




Angular Typescript

Marzo 2020

minsa1t



An Indra company

TypeScript

01

Características

El lenguaje TypeScript fue ideado por Anders Hejlsberg—autor de Turbo Pascal, Delphi, C# y arquitecto principal de .NET—como un supraconjunto de JavaScript que permitiese utilizar tipos estáticos y otras características de los lenguajes avanzados como la Programación Orientada a Objetos.

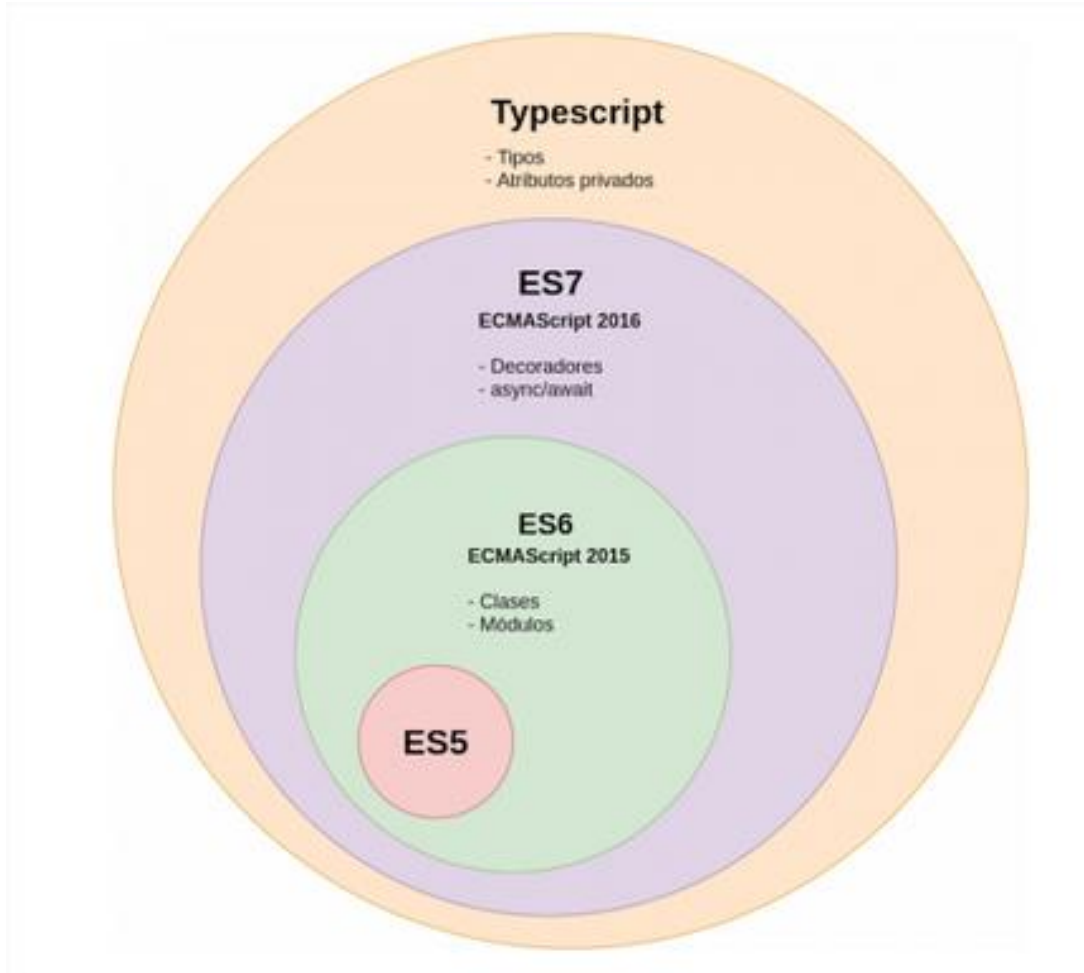
- TypeScript es un lenguaje Open Source basado en JavaScript y que se integra perfectamente con otro código JavaScript, solucionando algunos de los principales problemas que tiene JavaScript:
 - Falta de tipado fuerte y estático.
 - Falta de “Syntactic Sugar” para la creación de clases
 - Falta de interfaces
- TypeScript es un lenguaje con una sintaxis bastante parecida a la de C# y Java, por lo que es fácil aprender TypeScript para los desarrolladores con experiencia en estos lenguajes y en JavaScript.

Características

TypeScript se convierte en JavaScript perfectamente ejecutable en todos los navegadores actuales (y antiguos), pero se alcanza fácilmente:

- abordar desarrollos de gran complejidad,
- organizando el código fuente en módulos,
- utilizando características de auténtica orientación a objetos, y
- disponer de recursos de edición avanzada del código fuente (Intellisense, compleción de código, “snippets”, etc.), tanto en Visual Studio, como en otros editores populares, como Sublime Text, Eclipse, WebStorm, etc., cosa impensable hasta este momento, pero factible ahora mismo gracias a los “plug-in” de TypeScript disponibles para estos editores.

Características



- TypeScript se “transpila” a JavaScript cumpliendo el estándar ECMAScript totalmente compatible con las versiones existentes.
- De hecho, el “transpilador” deja elegir la versión ECMAScript del resultado, permitiendo adoptar la novedades mucho antes de que los navegadores las soporten: basta con dejar el resultado en la versión mas ampliamente difundida.

Características

- Amplia la sintaxis del JavaScript con la definición y comprobación de:
 - Tipos básicos: booleans, number, string, Any y Void.
 - Clases e interfaces
 - Herencia
 - Genéricos
 - Módulos
 - Anotaciones
- Otra de las ventajas que otorga Typescript es que permite comunicarse con código JavaScript ya creado e incluso añadirle “tipos” a través de unos ficheros de definiciones .d.ts que indican los tipos que reciben y devuelven las funciones de una librería.
- Ya existen ficheros de definiciones para la mayoría de los Framework mas populares (<http://definitelytyped.org/>).

IDEs soportados

- Visual Studio 2012 ... –native (+msbuild)
- Visual Studio Code
- ReSharper–included
- Sublime Text 2/3 –official plugin as a part of 1.5 release
- Online Cloud9 IDE
- Eclipse IDE
- IntelliJ IDEA
- Grunt, Maven, Gradleplugins

Tipos básicos

- **Boolean:**

```
var isDone: boolean= false;
```

- **Number:**

```
var height: number= 6;
```

- **String:**

```
varname: string= "bob";
```

- **Array:**

```
var list:number[] = [1, 2, 3];  
var list:Array<number> = [1, 2, 3];
```

- **Enum:**

```
enum Color {Red, Green, Blue};  
var c: Color = Color.Green;
```

- **Any:** var notSure: any = 4;

```
notSure= "maybea stringinstead";  
notSure= false; // okay, definitely a boolean  
var list:any[] = [1, true, "free"];
```

- **Void:**

```
function warnUser(msg: string): void {  
    alert(msg);  
}
```

- **Null y Undefined:**

- activar `--strictNullChecks` impide asignarlo a tipos simples, evitando errores comunes.
- Si se desea pasar en un tipo, nulo o indefinido, se puede utilizar el tipo unión: `tipo | null | undefined`.

- **Never:**

```
function error(message: string): never {  
    throw new Error(message);  
}
```


Tipos básicos

```
// 1 - declaracion del tipo
type Ranking = [number, string, boolean];

// 2 - definición de variables
let position: number;
let playerName: string;
let finishedGame: boolean;
let ranking: Ranking;
let hallOfFame: Array<Ranking> = [];

// 3 - crea un ranking
position = 1;
playerName = "Bruno Krebs";
finishedGame = true;
ranking = [position, playerName, finishedGame];
hallOfFame.push(ranking);
```

```
// 4 - crea otro ranking
position = 2;
playerName = "Maria Helena";
finishedGame = true;
ranking = [position, playerName, finishedGame];
hallOfFame.push(ranking);

// 5 - define una funcion que recorre todos los rankings
function printRankings(rankings: Array<RankingTuple>): void {
  for (let ranking of rankings) {
    console.log(ranking);
  }
}

// 6 - llama a la función
printRankings(hallOfFame);
```

Funciones (I)

- Tipos para los parámetros y el valor de retorno:

```
function add(x: number, y: number): number { return x+y; }
```

- Valores por defecto: Se pueden definir valores por defecto a los parámetros en las funciones.

```
function(valor = "foo") {...};
```

- Resto de los parámetros: Convierte una lista de parámetros en un array de Any.

```
function f (x: number, y: number, ...a): number {  
    return (x + y) * a.length  
}
```

```
f(1, 2, "hello", true, 7) === 9
```

- Operador de propagación: Convierte un array o cadena en una lista de parámetros.

```
Var str= "foo"  
Var chars= [ ...str] // [ "f", "o", "o" ]
```

Funciones (II)

- Tipos de parámetros:
 - Obligatorios

```
function buildName(firstName: string, lastName: string) {  
    return firstName + " " + lastName;  
}  
let result1 = buildName("Bob"); // error, too few parameters  
let result2 = buildName("Bob", "Adams", "Sr."); // error, too many parameters  
let result3 = buildName("Bob", "Adams"); // ah, just right
```

- Opcionales

```
function buildName(firstName: string, lastName?: string) {  
    if (lastName)  
        return firstName + " " + lastName;  
    else  
        return firstName;  
}  
let result1 = buildName("Bob"); // works correctly now  
let result2 = buildName("Bob", "Adams", "Sr."); // error, too many parameters  
let result3 = buildName("Bob", "Adams"); // ah, just right
```

Funciones (II)

- Tipos de parámetros:
 - Por defecto

```
function buildName(firstName: string, lastName = "Smith") {  
    return firstName + " " + lastName;  
}  
let result1 = buildName("Bob"); // works correctly now, returns "Bob Smith"  
let result2 = buildName("Bob", undefined); // still works, also returns "Bob Smith"  
let result3 = buildName("Bob", "Adams", "Sr."); // error, too many parameters  
let result4 = buildName("Bob", "Adams"); // ah, just right
```

- Orden adecuado

```
function buildName(firstName: string, lastName = "Smith", aka?: string)
```

Funciones (II)

- Uniones

```
1 function f(x: number | number[]) {
2     if (typeof x === "number") {
3         //x is a number here
4         return x + 10;
5     } else {
6         //x is a number[] here
7         let sum = 0;
8
9         x.forEach((element) => {
10             sum += element;
11         });
12
13         return sum;
14     }
15 }
16
17 f(10);
18 f([10, 20]);
```

Funciones (III)

- Funciones anónimas.

```
// Named function
```

```
function add(x, y) { return x + y; }
```

```
// Anonymous function
```

```
let myAdd = function(x, y) { return x + y; };
```

- Funciones lambda

```
let myAdd: (baseValue: number, increment: number) => number = function(x: number, y: number): number { return x + y; };
```

Estructuras y diseño

- Clases

```

1 class Person {
2     age: number;
3     name: string;
4     surname: string;
5
6     constructor(age: number,
7         name: string,
8         surname: string) {
9         this.age = age;
10        this.name = name;
11        this.surname = surname;
12    }
13
14    getFullName() {
15        return this.name + ' ' + this.surname;
16    }
17 }
18
19 var cadaver = new Person(60, "John", "Doe");
20
21 var boy = Person(10, "Vasya", "Utkin"); //compile error

```

VS

```

1 function Person(age, name, surname) {
2     this.age = age;
3     this.name = name;
4     this.surname = surname;
5 }
6
7 Person.prototype.getFullName = function () {
8     return this.name + " " + this.surname;
9 }
10
11 var cadaver = new Person(60, "John", "Doe");
12
13 var boy = Person(10, "Vasya", "Utkin"); //bad

```

Estructuras y diseño

- Constructores

```
1 class Person {  
2     constructor(  
3         public age: number,  
4         public name: string,  
5         public surname: string) {  
6     }  
7  
8     getFullName() {  
9         return this.name + ' ' + this.surname;  
10    }  
11 }  
12  
13 var cadaver = new Person(60, "John", "Doe");  
14  
15 var boy = Person(10, "Vasya", "Utkin"); //compile error
```



ES 5, por defecto todos son públicos, si se quieren privados se debe indicar explícitamente.

Estructuras y diseño

- Propiedades

```
1 class Person {  
2     constructor(  
3         private age: number,  
4         private name: string,  
5         private surname: string) {  
6     }  
7  
8     get fullName() {  
9         return this.name + " " + this.surname;  
10    }  
11 }  
12  
13 var cadaver = new Person(60, "John", "Doe");  
14 var name = cadaver.fullName;
```

Estructuras y diseño

- Iterador for ... of vs for ... in

```
let list = [4, 5, 6];
```

```
for (let i in list) {  
  console.log(i); // "0", "1", "2",  
}
```

```
for (let i of list) {  
  console.log(i); // "4", "5", "6"  
}
```

Estructuras y diseño

- Herencia

```
1 class Animal {
2     constructor(public name: string) { }
3     move(meters: number) {
4         alert(this.name + " moved " + meters + "m.");
5     }
6 }
7
8 class Snake extends Animal {
9     constructor(name: string) { super(name); }
10    move() {
11        alert("Slithering...");
12        super.move(5);
13    }
14 }
15
16 var sam = new Snake("Sammy the Python");
17
18 sam.move(1);
19
```

Estructuras y diseño

- Genéricos

```
1 class Greeter<T> {  
2     greeting: T;  
3     constructor(message: T) {  
4         this.greeting = message;  
5     }  
6     greet() {  
7         return this.greeting;  
8     }  
9 }  
10  
11 var greeter = new Greeter<string>("Hello, world");
```

Estructuras y diseño

- Interfaces

```
1 interface IMovable {  
2     move();  
3 }  
4  
5 class Chair implements IMovable {  
6     move() {  
7         console.log("Somebody moved me!");  
8     }  
9 }  
10  
11 function moveTwice(object: IMovable) {  
12     object.move();  
13     object.move();  
14 }  
15 |  
16 moveTwice(new Chair());
```

Estructuras y diseño

- Decoradores de clase

```
const log = (originalConstructor: new(...args: any[]) => T) => {  
  function newConstructor(... args) {  
    console.log("Argumentos: ", args.join(", "));  
    new originalConstructor(args);  
  }  
  
  newConstructor.prototype = originalConstructor.prototype;  
  return newConstructor;  
}  
  
@log  
class Person {  
  constructor(name: string, age: number) {}  
}  
  
new Person("Miguel", 33);  
//Argumentos: Miguel, 33
```

Estructuras y diseño

- Decoradores de método

```
1 class C {  
2   @readonly  
3   @enumerable(false)  
4   method() { }  
5 }  
6  
7 function readonly(target, key, descriptor) {  
8   descriptor.writable = false;  
9 }  
10  
11 function enumerable(value) {  
12   return function (target, key, descriptor) {  
13     descriptor.enumerable = value;  
14   }  
15 }
```

Estructuras y diseño

- Angular 2.0 + TypeScript

```
1 import {Component, View, bootstrap}
2 from 'angular2/angular2';
3
4 @Component({
5   selector: 'my-app'
6 })
7 @View({
8   template: '<h1>Hello {{name}}</h1>'
9 })
10 class MyAppComponent{
11   name: string;
12   constructor() {
13     this.name = 'Alice'
14   }
15 }
16
17 bootstrap(MyAppComponent);
```


Angular 8+

02

Introducción

- Angular es un Framework estructural para aplicaciones web de una sola página (SPA) siguiendo el patrón MVVM y orientación a componentes.
- Permite utilizar el HTML como lenguaje de plantillas y extender la sintaxis del HTML.
- El enlazado de datos y la inyección de dependencias eliminan gran parte del código.
- Define claramente la separación entre el modelo de datos, el código, la presentación y la estética.
- Las reglas de Angular permiten eliminar las carencias de libertinaje de JavaScript.
- Angular es la evolución de AngularJS aunque incompatible con su predecesor.



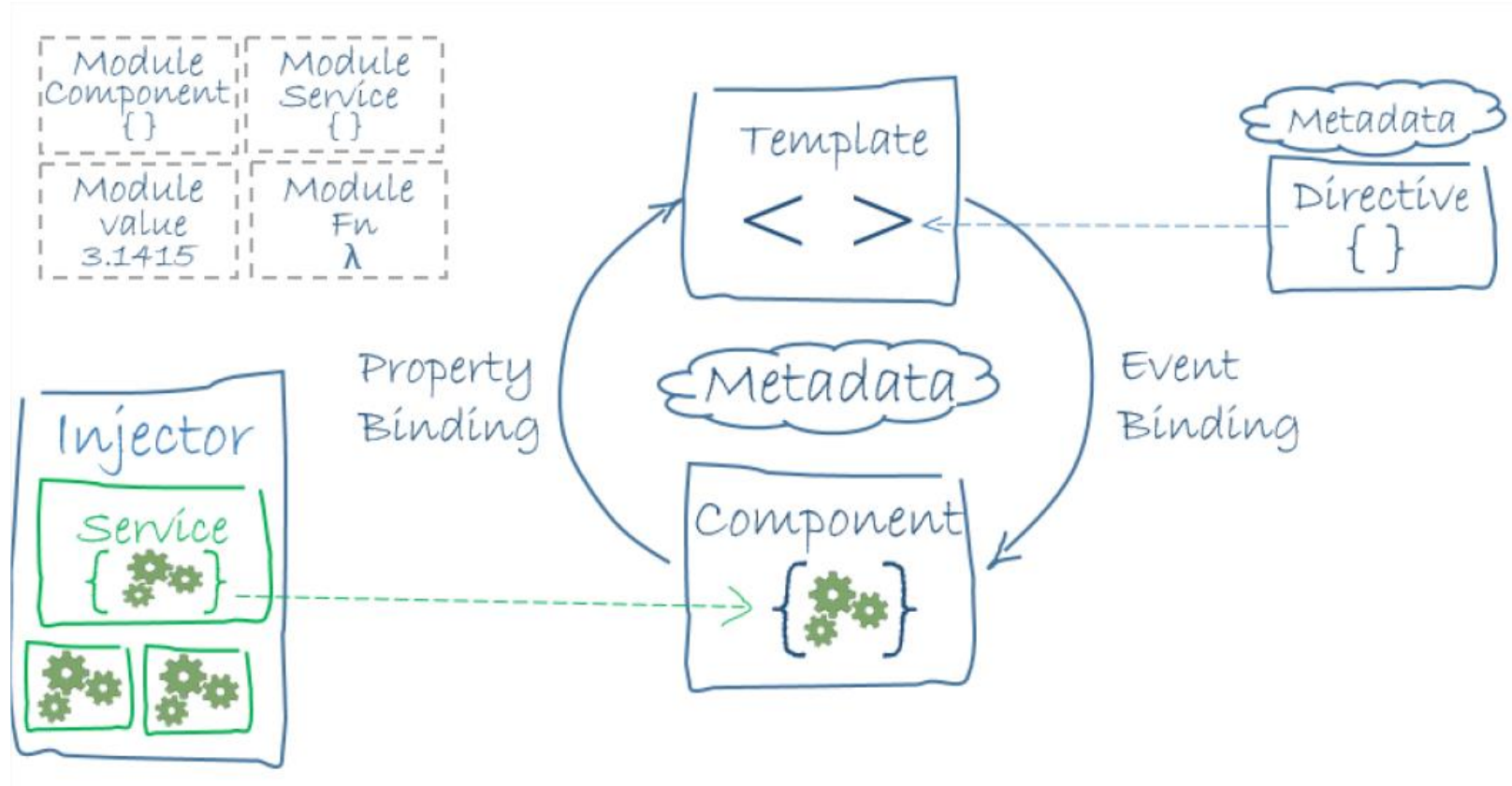
Introducción

Angular utiliza el patrón **Single-Page Application (SPA)** o aplicación de página única.

Un single-page application (SPA), o aplicación de página única es una aplicación web o es una aplicación web que utiliza **una sola página** con el propósito de dar una **experiencia más fluida** a los usuarios como una aplicación de escritorio.

Todo el código HTML, Javascript y CSS se carga de **una sola vez**
Los recursos necesarios **se cargan dinámicamente** cuando lo requiera la página

Introducción



Estructuras Angular

Bibliotecas @Angular:

- Los contenedores Angular son una colección de módulos de JavaScript, se puede pensar en ellos como módulos de biblioteca.
- Cada nombre de la biblioteca Angular comienza con el prefijo @angular.
- Se pueden instalar con NPM e importar partes de ellos con instrucciones import de JavaScript.
- De esta manera se está utilizando tanto los sistemas de módulos de Angular como los de JavaScript juntos.

Estructuras Angular

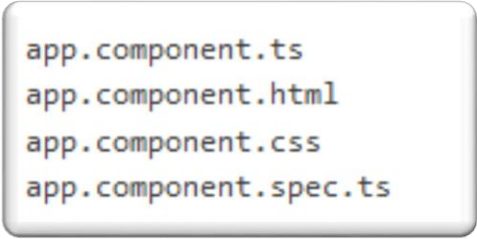
Módulos:

- Las aplicaciones Angular son modulares.
- Angular dispone de su propio sistema de módulos (NgModules).
- Cada aplicación Angular tiene al menos un módulo, el módulo raíz, convencionalmente denominado AppModule.
- Mientras que el módulo de raíz puede ser el único módulo en una aplicación pequeña, en la mayoría de las aplicaciones existirán muchos más “módulos de características”, cada uno como un bloque coherente de código dedicado a un dominio de aplicación, un flujo de trabajo o un conjunto estrechamente relacionado de capacidades.
- El módulo Angular, una clase decorada con @NgModule, es una característica fundamental de Angular.

Estructuras Angular

Componentes:

- Un componente controla una parte de la pantalla denominada vista.
- Se puede considerar que un componente es una clase con interfaz de usuario.
- Reúne en un único elemento: el código (clase que expone datos y funcionalidad), su representación (plantilla HTML) y su estética (CSS).
- El código interactúa con la plantilla a través del enlazado dinámico.
- Una vez creado, un componente es como una nueva etiqueta HTML que puede ser utilizada en otras plantillas.
- Angular crea, actualiza y destruye los componentes según el usuario se mueve a través de la aplicación.
- Angular permite personalizar mediante el uso de enganches (hooks) lo que pasa en cada fase del ciclo de vida del componente.



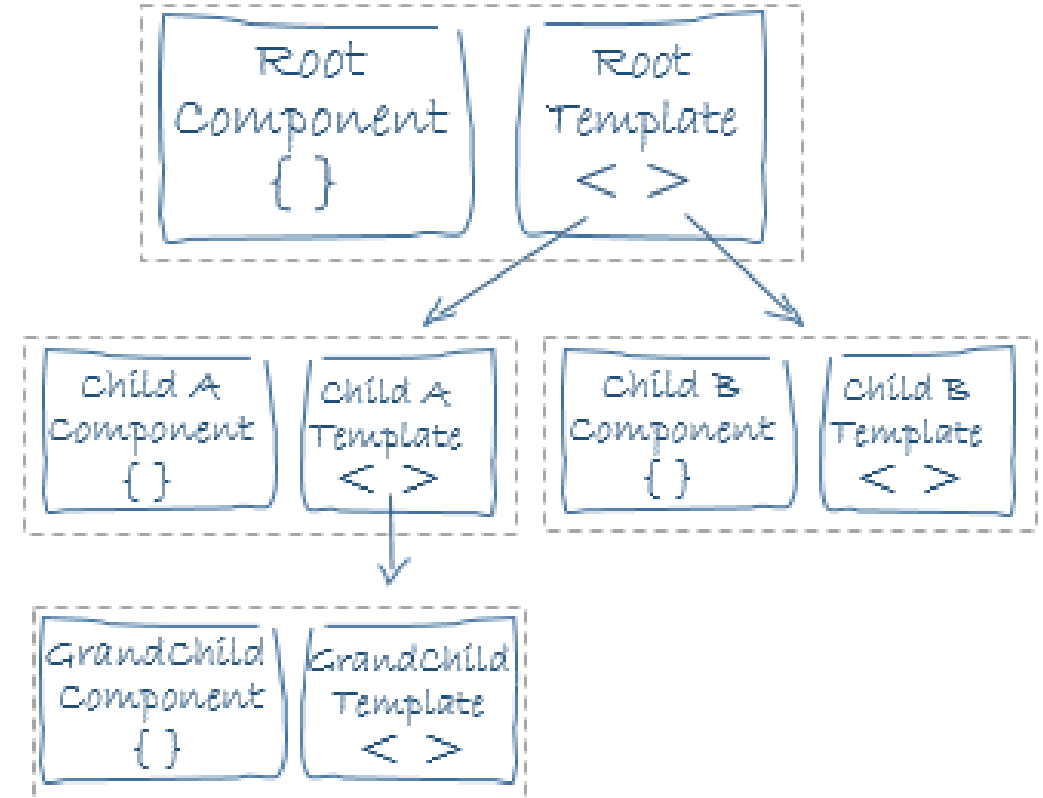
```
app.component.ts  
app.component.html  
app.component.css  
app.component.spec.ts
```

Estructuras Angular

Plantillas (I)

- Las plantillas contienen la representación visual de los componentes.
- Son segmentos escritos en HTML ampliado con elementos suministrados por Angular como las directivas, marcadores, pipes, ...
- Cuentan con la lógica de presentación pero no permiten la codificación: los datos y la funcionalidad se los suministra la clase del componente.

```
app.component.ts
app.component.html
app.component.css
app.component.spec.ts
```



- Una plantilla puede utilizar otros componentes, actúan como contenedores, dichos componentes pueden contener a su vez otros componentes. Esto crea un árbol de componentes: el modelo de composición.

Estructuras Angular

Plantillas (II) - Directivas

Las directivas son marcas en los elementos del árbol DOM, en los nodos del HTML, que nos permiten añadir comportamiento dinámico al árbol DOM.

Se pueden usar directivas propias de Angular o extender la funcionalidad hasta donde necesitemos creando las nuestras propias.

Las directivas mas usadas propias de Angular son:

- ***ngIf, *ngFor, *ngSwitch, ng-container, etc.**
- **ngStyle:** permite gestionar el estilo dinámicamente.
- **ngClass:** simplifica el proceso cuando se manejan multiples clases CSS que aparecen y desaparecen.
- **ngModel (acompañado de name)**

Estructuras Angular

Plantillas (III) - Pipes

El resultado de una expresión puede requerir alguna transformación antes de estar listo para mostrarla en pantalla: mostrar un número como moneda, que el texto pase a mayúsculas o filtrar una lista y ordenarla.

Los Pipes de Angular, anteriormente denominados filtros, son simples funciones que aceptan un valor de entrada y devuelven un valor transformado. Son fáciles de aplicar dentro de expresiones de plantilla, usando el operador tubería (|).

Son una buena opción para las pequeñas transformaciones, dado que al ser encadenables un conjunto de transformaciones pequeñas pueden permitir una transformación compleja.

Estructuras Angular

Servicios

Los servicios en Angular son una categoría muy amplia que abarca cualquier valor, función o característica que necesita una aplicación.

Es una clase con un propósito limitado y bien definido, que debe hacer algo específico y hacerlo bien.

La responsabilidad de un componente es implementar la experiencia de los usuarios y nada más delega todo lo demás en las directivas (manipulación del DOM) y los servicios (lógica de negocio).

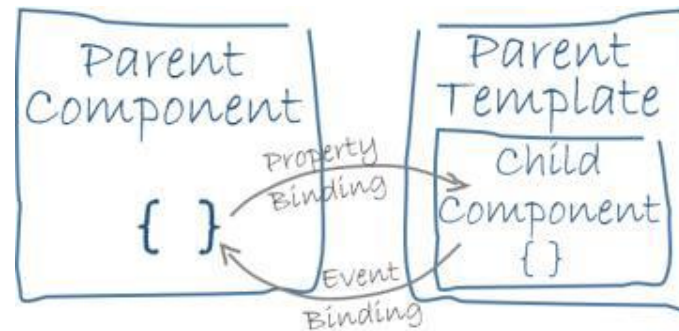
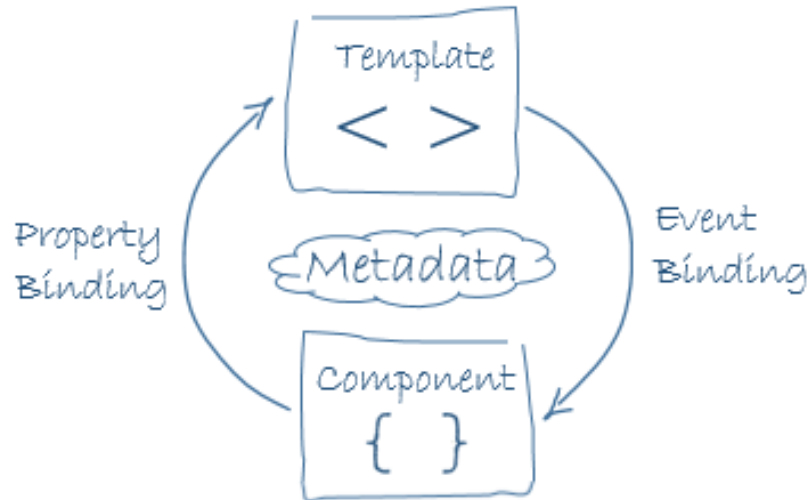
Conceptos Angular

Enlace de datos (I)

Automatiza, de una forma declarativa, la comunicación entre los controles HTML de la plantilla y las propiedades y métodos del código del componente.

Requieren un nuevo modelo mental declarativo frente al imperativo tradicional.

El enlace de datos también permite la comunicación entre los componentes principales y secundarios.

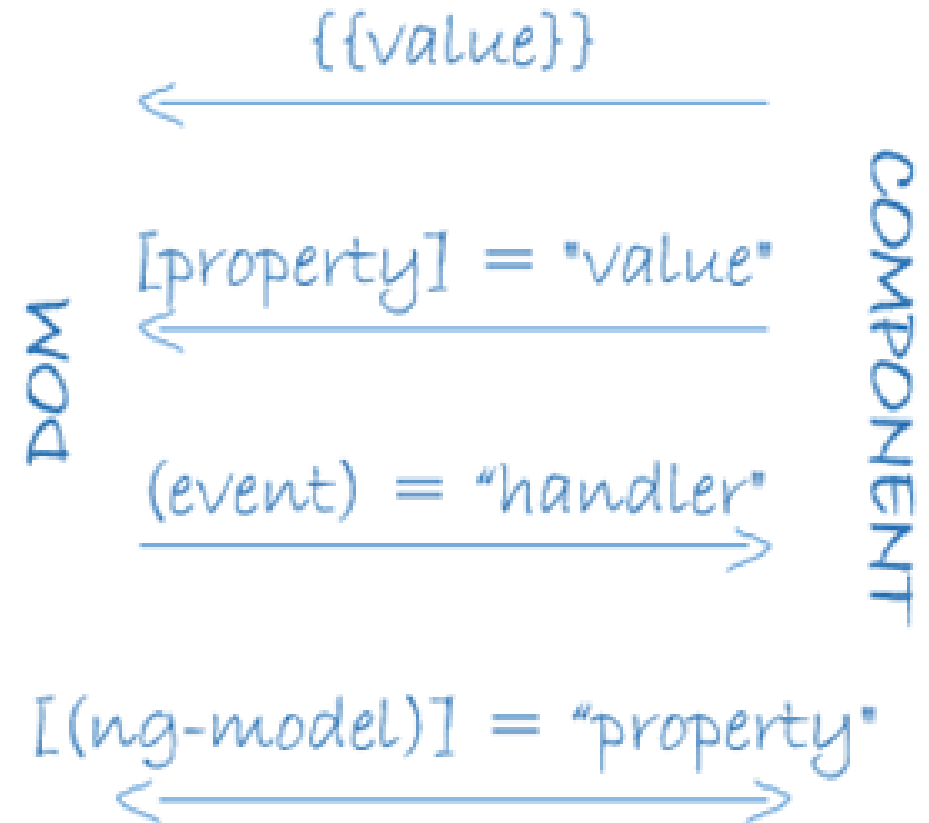


Conceptos Angular

Enlace de datos (II).

La comunicación entre Componente y Plantilla puede ser:

- Unidireccional:
 - Datos (C→P)
 - Interpolación
 - Propiedades
 - Comandos (P→C)
 - Eventos
- Bidireccional:
 - Datos (P↔C)
 - Directivas



Conceptos Angular

Inyección de dependencia.

La Inyección de Dependencias (DI) es un mecanismo que proporciona nuevas instancias de una clase con todas las dependencias que requiere plenamente formadas.

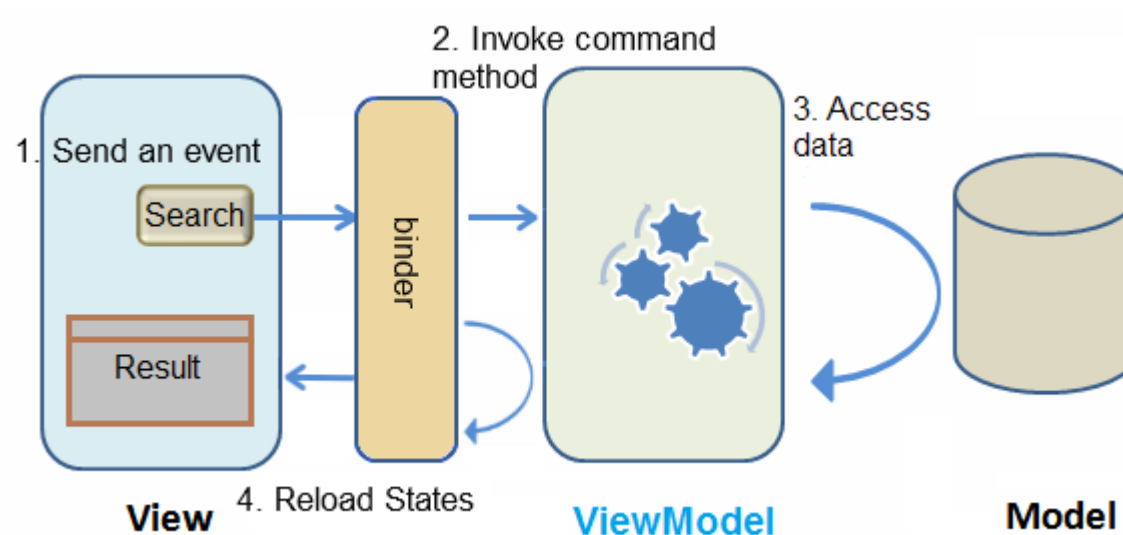
Gracias a TypeScript, Angular sabe de qué servicios depende un componente con tan solo mirar su constructor.

El Injector es el principal mecanismo detrás de la DI. A nivel interno, un inyector dispone de un contenedor con las instancias de servicios que crea él mismo. Si una instancia no está en el contenedor, el inyector crea una nueva y la añade al contenedor antes de devolver el servicio a Angular, por eso los servicios son singletons en el ámbito de su inyector.

Conceptos Angular

MVVM

- El **Modelo** es la entidad que representa el concepto de negocio.
- La **Vista** es la representación gráfica del control o un conjunto de controles que muestran el Modelo de datos en pantalla.
- La **VistaModelo** es la que une todo. Contiene la lógica del interfaz de usuario, los comandos, los eventos y una referencia al Modelo.



Desarrollo

03

Entorno desarrollo Angular (I)

- Node.js.

Entorno en tiempo de ejecución

- Descargar e instalar: <https://nodejs.org>
- Verificar desde consola de comandos:
`node -v`

- Npm.

Aunque se instala con Node.js es conveniente actualizarlo:

```
npm update -g npm
```

Configuración:

```
npm config edit
```

```
proxy=http://usr:pwd@proxy.dominion.com:8080
```

- Angular CLI (*Command Line Interface*)

Permite generar proyectos y plantillas de código desde consola, así como ejecutar un servidor de desarrollo o lanzar los tests de la aplicación. (<https://cli.angular.io/>)

```
npm install -g @angular/cli@latest
```

Entorno desarrollo Angular (II)

- Typescript:
`npm install -g typescript`
- Visual Studio Code:
<https://code.visualstudio.com/download>
- Extensiones:
 - JavaScript (ES6) code snippets
 - Prettier – Code Formatter
 - JSHint
 - Angular Snippets (Version 9)
 - Visual Studio IntelliCode
- Configuración de ayuda:
 - Ctrl + Mayúsculas + P → Settings (UI) → Escribir “format” → Check “Editor:Format On Save”

Directivas Angular CLI (I)

Hay elementos de código que mantienen una cierta estructura y no necesitas escribirla cada vez que creas un archivo nuevo.

El comando *generate* permite generar algunos de los elementos habituales en Angular:

- Componentes: `ng generate component my-new-component`
- Directivas: `ng g directive my-new-directive`
- Pipe: `ng g pipe my-new-pipe`
- Servicios: `ng g service my-new-service`
- Clases: `ng g class my-new-class`
- Interface: `ng g interface my-new-interface`
- Enumerados: `ng g enum my-new-enum`
- Módulos: `ng g module my-module`

Directivas Angular CLI (II)

- Construye la aplicación en la carpeta /dist.

```
ng build  
ng build--dev
```

- Paso a producción, construye optimizándolo todo para producción.

```
ng build --prod  
ng build --prod --env=prod  
ng build --target=production --environment=prod
```

- Precompilala aplicación

```
ng build--prod--aot
```

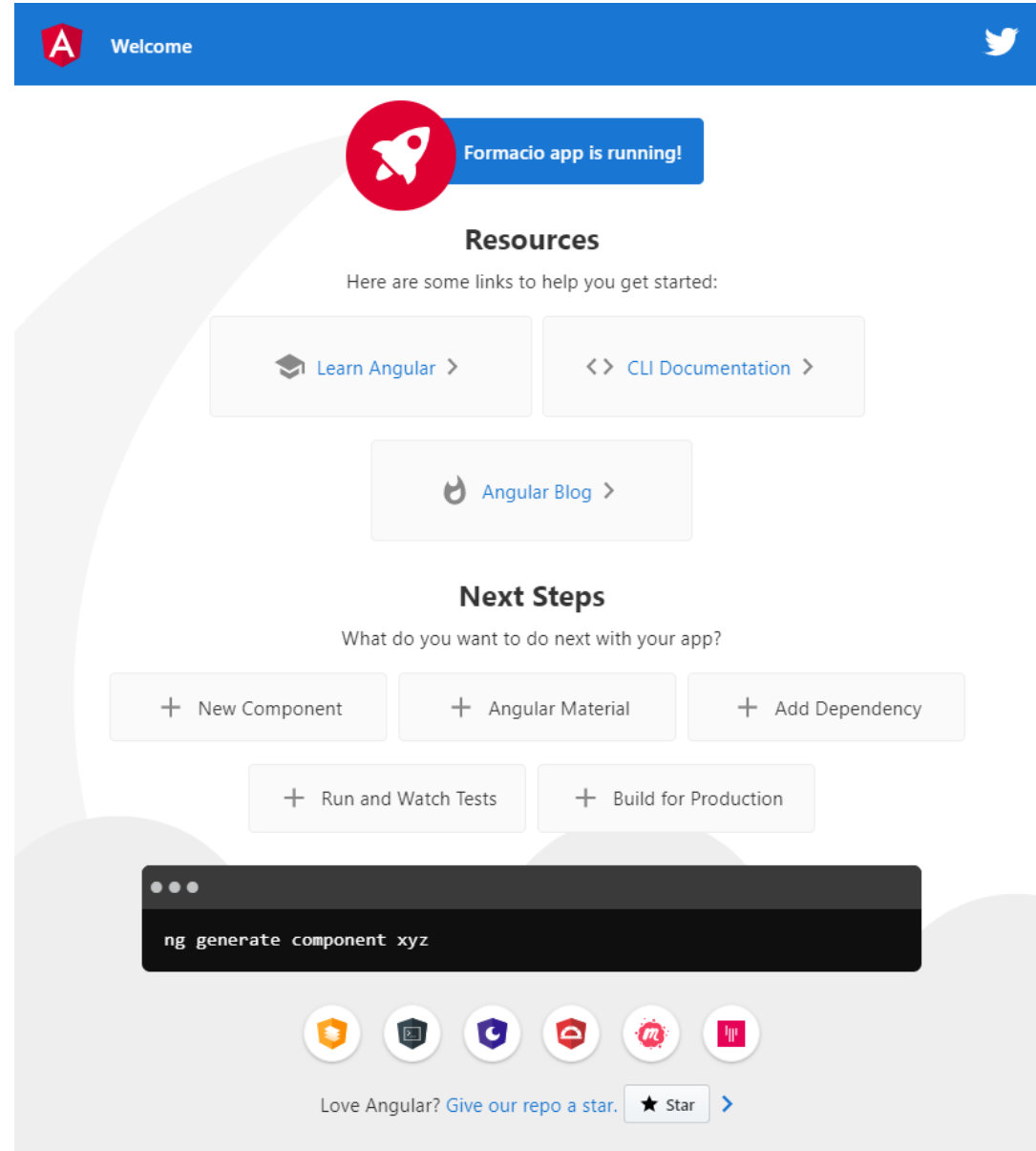
Directivas Angular CLI (III)

- Crear un nuevo proyecto Angular

`ng new Formacio`

- Arrancar un proyecto

`ng serve`



Estructura proyecto Angular

-  src
-  e2e
-  .editorconfig
-  .gitignore
-  angular.json
-  karma.conf.js
-  package.json
-  README.md
-  tsconfig.json
-  tslint.json

- src: Fuentes de la aplicación
- e2e: Test endto end
- Ficheros de configuración:
 - .editorconfig : Configuración de <https://editorconfig.org/>
 - .gitignore: archivo de git
 - angular.json: configuración del proyecto.
 - karma.conf.js: configuración de los tests del proyecto.
 - package.json: enunciado de dependencias angular.
 - README.md: archivo instrucciones github.
 - tsconfig.json: configuración del compilador TypeScript.
 - tslint.json: configuración tslint (guía de estilos).

Estructura proyecto Angular

src

app

- app-routing.module.ts
- app.component.css
- app.component.html
- app.component.spec.ts
- app.component.ts
- app.module.ts

assets

environments

- environment.prod.ts
- environment.ts

✦ favicon.ico

- index.html
- main.ts
- polyfills.ts
- styles.css
- test.ts

- app: Carpeta que contiene los ficheros fuente principales de la aplicación.
- assets: Carpeta con los recursos que se copiarán a la carpeta build.
- environments: configuración de los diferentes entornos.
- favicon.ico: Icono para el navegador.
- index.html: Página principal (SPA).
- main.ts: Arranque del módulo principal de la aplicación. No es necesario modificarlo.
- polyfills.js: importación de los diferentes módulos de
- style.css: Se editará para incluir CSS global de la web, se concatena, minimiza y enlaza en index.html automáticamente.
- test.ts: pruebas del arranque. compatibilidad ES6 y ES7.

Tour of Heroes

<https://angular.io/tutorial>

Tour of Heroes - Componentes

```
import { Component, OnInit } from '@angular/core';
```

```
@Component(  
  {  
    selector: 'app-heroes',  
    templateUrl: './heroes.component.html',  
    styleUrls : ['./heroes.component.css']  
  })
```

```
export class HeroesComponent implements OnInit {  
  
  constructor() {  
  }  
  
  ngOnInit() {  
  }  
}
```

Tour of Heroes – Interfaz Hero



ng generate component components/heroes

ng g c components/heroes

Tour of Heroes – Interfaz Hero



ng generate interface interfaces/Hero

ng g i interfaces/Hero

Tour of Heroes – Capitalize Pipe



ng g p pipes/capitalize

```
import { Pipe, PipeTransform } from "@angular/core";
import { stringify } from "querystring";

@Pipe({
  name: "capitalize"
})
export class CapitalizePipe implements PipeTransform {
  transform(value: string, ...args: any[]): any {
    let capitalized = "";
    value.split(" ").forEach(word => {
      if (word[0] === word[0].toUpperCase()) {
        capitalized = capitalized + word;
      } else {
        capitalized = capitalized + word[0].toUpperCase() + word.slice(1);
      }
    });
    return null;
  }
}
```

Tour of Heroes – Estructura del módulo principal



Module Structure

```
import { BrowserModule } from '@angular/platform-browser';
import { NgModule } from '@angular/core';

import { AppRoutingModule } from './app-routing.module';
import { AppComponent } from './app.component';
import { CapitalizePipe } from './pipes/capitalize.pipe';
import { HeroesComponent } from './components/heroes/heroes.component';
;
import { FormsModule } from "@angular/forms";

@NgModule({
  declarations: [
    AppComponent,
    CapitalizePipe,
    HeroesComponent
  ],
  imports: [
    BrowserModule,
    AppRoutingModule,
    FormsModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Tour of Heroes – Decorador @Input



@Input

```
import { Component, OnInit, Input } from '@angular/core';
import { Hero } from "src/app/interfaces/hero";

@Component({
  selector: 'app-hero-detail',
  templateUrl: './hero-detail.component.html',
  styleUrls: ['./hero-detail.component.css']
})
export class HeroDetailComponent implements OnInit {
  @Input() hero: Hero;

  constructor() { }

  ngOnInit() {
  }
}
```

Tour of Heroes – Añadir un componente



heros.component.html



[hero]

hero-detail.component.ts

```
<h2>My Heroes</h2>
<ul class="heroes">
  <li
    *ngFor="let hero of heroes"
    [ngClass]="{ selected: hero === selectedHero }"
    (click)="onSelect(hero)"
  >
    <span class="badge">{{ hero.id }}</span> {{ hero.name }}
  </li>
</ul>

<app-hero-detail [hero]="selectedHero"></app-hero-detail>
```

Tour of Heroes – Estructura de un servicio



hero.service.ts

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class HeroService {

  constructor() { }

}
```


Tour of Heroes – Observables (RxJS)



Observable & Subscribe

```
import { Observable, of } from 'rxjs';
```

```
.  
.br/.
```

```
getHeroes(): Observable<Hero[]> {  
  return of(HEROES);  
}
```

```
.. .. ..
```

```
getHeroes(): void {  
  this.heroService.getHeroes()  
    .subscribe(heroes => (this.heroes = heroes));  
}
```

Tour of Heroes – Notacion string con parámetro



Notación ``

```
this.messageService.add(`HeroService: Selected hero id=${hero.id}`);
```

Tour of Heroes – Gestión de errores



catchError

```
getHero(id: number): Observable<Hero> {  
  const url = `${this.heroesUrl}/${id}`;  
  return this.http.get<Hero>(url).pipe(  
    tap(_ => this.log(`fetched hero id=${id}`)),  
    catchError(this.handleError<Hero>(`getHero id=${id}`))  
  );  
}
```

```
private handleError<T> (operation = 'operation', result?: T) {  
  return (error: any): Observable<T> => {  
  
    // TODO: send the error to remote logging infrastructure  
    console.error(error); // log to console instead  
  
    // TODO: better job of transforming error for user consumption  
    this.log(`${operation} failed: ${error.message}`);  
  
    // Let the app keep running by returning an empty result.  
    return of(result as T);  
  };  
}
```

Tour of Heroes – Sincronía en llamadas asíncronas. RxJS operators.



switchMap vs flatMap

- *Subject* (tipo)
Observable de entrada/salida. Añade valores con “*next*”, obtiene valores con “*subscribe*”.
- *distinctUntilChanged()* & *switchMap()* (operadores)
 - *distinctUntilChanged()* emite un valor que es distinto al anterior.
 - *switchMap()* realiza llamadas a observables con valores de entrada. Solo emite la respuesta de la llamada mas reciente eliminando las anteriores.

Bootstrap – Instalación con npm para Angular CLI

- `npm install bootstrap`
 npm WARN bootstrap@4.4.1 requires a peer of **jquery@1.9.1 - 3** but none is installed. You must install peer dependencies yourself.
 npm WARN bootstrap@4.4.1 requires a peer of **popper.js@^1.16.0** but none is installed. You must install peer dependencies yourself.
- `npm install jquery`
- `npm install popper.js`



```

"projects": {
  "Formacio": {
    .
    .
    .
    "architect": {
      "build": {
        "builder": "@angular-devkit/build-angular:browser",
        "options": {
          "outputPath": "dist/Formacio",
          "index": "src/index.html",
          "main": "src/main.ts",
          "polyfills": "src/polyfills.ts",
          "tsConfig": "tsconfig.app.json",
          "aot": false,
          "assets": ["src/favicon.ico", "src/assets"],
          "styles": [
            "src/styles.css",
            "node_modules/bootstrap/dist/css/bootstrap.min.css"
          ],
          "scripts": [
            "node_modules/bootstrap/dist/js/bootstrap.min.js",
            "node_modules/jquery/dist/jquery.min.js",
            "node_modules/popper.js/dist/umd/popper.min.js"
          ]
        }
      }
    }
  }
}

```

¡Gracias!

Presentación:

Nombre Apellido Apellido
nombre@minsait.com

Avda. de Bruselas 35
28108 Alcobendas,
Madrid España

T +34 91 480 50 00
F +34 91 480 50 80
www.minsait.com

minsait

An Indra company

minsait

Mark Making the way forward

An Indra company